

```
# prediction.py
#
# Phase 1 confirmation-probability logic.
#
# IMPORTANT: this is a heuristic model, not
# a trained ML model – because we
# don't have historical pnr_history data
# yet (see architecture doc, section 5,
# "Cold start problem"). It replaces the
# frontend's random seed() function
# with numbers based on known, published
# general patterns in Indian Railways
# booking behaviour. Once real pnr_history
# rows accumulate in the database,
# swap `heuristic_confirmation_score` below
# for a trained
# sklearn.linear_model.LogisticRegression
# without changing the function
# signature – nothing else in this file or
# in main.py needs to change.

from datetime import date
from typing import Dict

# Rough, published-pattern-based base
# confirmation rates by class and quota.
# These are starting priors, not measured
# data – label results from these
# as "estimated" in the UI until real data
```

```

replaces them.
BASE_CONFIRM_RATE: Dict[str, float] = {
    "SL": 0.72,
    "3A": 0.68,
    "2A": 0.78,
    "1A": 0.90,
    "CC": 0.75,
    "EC": 0.85,
}

# Confirmation probability rises the
# earlier you book relative to the
# journey date. This is the single
# strongest real-world predictor.
def _days_before_multiplier(days_before:
int) -> float:
    if days_before >= 30:
        return 1.15
    if days_before >= 15:
        return 1.08
    if days_before >= 7:
        return 1.0
    if days_before >= 3:
        return 0.85
    if days_before >= 1:
        return 0.65
    return 0.45 # booking on the day of /
Tatkal-adjacent

def _weekday_multiplier(journey_date_str:
str) -> float:
    try:
        d =

```

```

date.fromisoformat(journey_date_str)
except ValueError:
    return 1.0
    # Friday/Sunday (weekday() 4 and 6) see
the heaviest demand in India.
weekday = d.weekday()
if weekday in (4, 6):
    return 0.90
if weekday in (5,): # Saturday
    return 0.95
return 1.05

def heuristic_confirmation_score(
    travel_class: str,
    journey_date_str: str,
    days_before_journey: int,
    train_popularity_factor: float = 1.0,
) -> int:
    """
    Returns an integer 0-100 confirmation
    probability.

    train_popularity_factor: pass < 1.0 for
    known high-demand trains (e.g.
    an average historical confirm-rate
    lower than the class baseline), or
    > 1.0 for trains that reliably confirm.
    Defaults to neutral (1.0) until
    per-train historical stats exist in
    pnr_history.
    """
    base =
    BASE_CONFIRM_RATE.get(travel_class, 0.70)

```

```

        score = (
            base
            *
            _days_before_multiplier(days_before_journey
            )
            *
            _weekday_multiplier(journey_date_str)
            * train_popularity_factor
        )
        return max(2, min(98, round(score *
100)))

def
recommendation_score(confirm_probability:
int, duration_minutes: int, fare: float) ->
int:
    """
        Combines confirmation probability with
a light penalty for very long
        journeys and very high fares, to rank
trains for "fastest & cheapest"
        style recommendations. Weights are
intentionally simple for Phase 1 –
        revisit once we have user click/booking
feedback to tune against.
    """
    duration_penalty = min(20,
duration_minutes / 60) # up to -20 pts for
very long trips
    fare_penalty = min(15, fare / 500) #
up to -15 pts for very expensive tickets
    score = confirm_probability -

```

```
duration_penalty * 0.3 - fare_penalty * 0.3  
    return max(0, min(100, round(score)))
```